SIGMA

Manual paso a paso para visualizar y editar SQLite en Expo Go, celular o emulador

Aplicación: SIGMA Opción 2 - Express + MySQL + SQLite Offline First

Objetivo: inspeccionar las tablas locales, ejecutar consultas y probar la cola de sincronización.

1. Qué se va a inspeccionar

La app de SIGMA crea una base SQLite en el celular o emulador. Esta base persiste aunque se cierre la aplicación y permite que el trabajo continúe sin conexión.

Elemento	Nombre en el ejemplo	Función
Base de datos	sigma-offline-first.db	Archivo SQLite local creado por expo-sqlite.
Tabla principal	tasks	Tareas que se muestran en la app.
Tabla de cola	sync_queue	Cambios CREATE, UPDATE o DELETE todavía no confirmados por el servidor.
Inspector	Open expo-sqlite	Herramienta visual de Expo para navegar tablas, editar filas y ejecutar SQL.

Antes de empezar

La app debe estar abierta en Expo Go, en un emulador Android o en el simulador iOS. El comando de Expo tiene que seguir ejecutándose en una terminal.

La edición manual cambia los datos reales de esa instalación de la app. Para hacer pruebas, usar tareas de ejemplo o datos que se puedan eliminar.

2. Abrir el inspector de SQLite

Seguir estos pasos en orden:

Paso	Acción
1	Abrir una terminal dentro de la carpeta `mobile` del proyecto.
2	Iniciar Expo con `npx expo start`.
3	Abrir SIGMA en el celular con el QR, o usar el emulador/simulador.
4	Confirmar que la pantalla de tareas se ve en el dispositivo.
5	Volver a la terminal de Expo y presionar `Shift + M`.
6	En el menú ampliado, elegir `Open expo-sqlite`.
7	Se abrirá una pestaña del navegador. Seleccionar la base `sigma-offline-first.db`.
8	Elegir una tabla o abrir el editor de consultas SQL.

Comando inicial

```
cd mobile
npx expo start
```

Si no aparece la opción Open expo-sqlite, verificar que la app está conectada al servidor de Expo y que se abrió al menos una vez. Las tablas se crean al iniciar la app.

3. Explorar las tablas

Una vez seleccionada la base, usar la barra lateral del inspector para abrir sus tablas. La vista de filas permite revisar y editar valores sin escribir SQL.

Tabla `tasks`

Campo	Significado
id	Identificador único de la tarea.
title	Texto visible en la lista de tareas.
status	PENDIENTE o FINALIZADA.
updated_at	Fecha y hora del último cambio local.
deleted_at	Fecha de eliminación lógica; si es NULL, la tarea sigue visible.
sync_status	PENDIENTE, SINCRONIZADA o ERROR.

Tabla `sync_queue`

Campo	Significado
id	Número interno de la operación encolada.
type	CREATE, UPDATE o DELETE.
task_id	Id de la tarea afectada.
payload	JSON que será enviado a Express cuando se sincronice.
created_at	Momento en que se agregó a la cola.

4. Consultas SQL básicas

Pegar cada consulta en el editor SQL del inspector y ejecutar. Las consultas SELECT solo leen datos; no modifican nada.

Ver todas las tareas

```
SELECT *
FROM tasks;
```

Ver tareas en el orden más reciente

```
SELECT id, title, status, sync_status, updated_at
FROM tasks
ORDER BY updated_at DESC;
```

Ver únicamente tareas pendientes de sincronización

```
SELECT id, title, status, sync_status
FROM tasks
WHERE sync_status = 'PENDIENTE';
```

Ver la cola de operaciones

```
SELECT id, type, task_id, payload, created_at
FROM sync_queue
ORDER BY id ASC;
```

Contar operaciones que faltan enviar

```
SELECT COUNT(*) AS operaciones_pendientes
FROM sync_queue;
```

5. Edición y pruebas de sincronización

Estas consultas sí modifican datos. Usarlas para comprobar visualmente el comportamiento offline-first.

Marcar una tarea como pendiente

```
UPDATE tasks
SET sync_status = 'PENDIENTE'
WHERE title = 'Revisar impresora del laboratorio';
```

Cambiar el estado de una tarea

```
UPDATE tasks
SET status = 'FINALIZADA',
sync_status = 'PENDIENTE'
WHERE title = 'Revisar impresora del laboratorio';
```

Eliminar todos los registros de prueba

```
DELETE FROM sync_queue;
DELETE FROM tasks;
```

Precaución

El último bloque borra todos los datos locales de esa instalación. Es útil para reiniciar una demostración, pero no recupera automáticamente las tareas que ya existían en MySQL. Luego hay que presionar **Sincronizar ahora** con conexión para descargar la copia central.

Prueba guiada recomendada

Momento	Qué observar
Sin conexión	Crear o modificar una tarea. `tasks` cambia y `sync_queue` gana una fila.
Antes de sincronizar	La tarea muestra PENDIENTE. La cola no está vacía.
Después de sincronizar	La cola se vacía y la tarea pasa a SINCRONIZADA.
Si falla la API	La fila queda en la cola y el estado puede pasar a ERROR.

6. Cierre y diagnóstico rápido

Situación	Qué revisar
No veo la base	Abrir la app primero; SQLite se inicializa durante el arranque.
No veo las tablas	Recargar la app y volver a abrir el inspector.
La cola no se vacía	Comprobar conexión, URL de `mobile/src/config.ts`, backend Express y MySQL.
Los datos no son los esperados	Consultar primero `tasks` y `sync_queue`; verificar si hay operaciones pendientes.
Quiero reiniciar	Usar los DELETE de la página anterior solo si se acepta perder los datos locales.

Idea principal

SQLite no es solo una caché en este ejemplo: es el lugar donde la app confirma el trabajo del usuario primero. `sync_queue` es la evidencia de lo que todavía debe llegar al servidor. El inspector permite demostrar ambos conceptos de forma visible en clase.